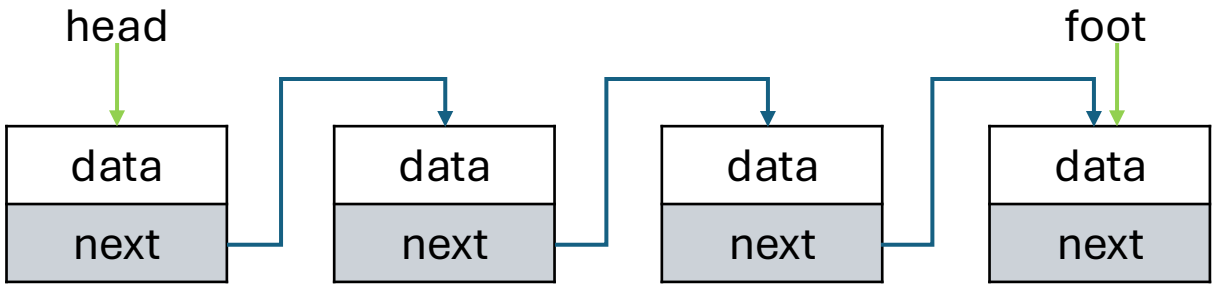
Hello Class!

Linked List

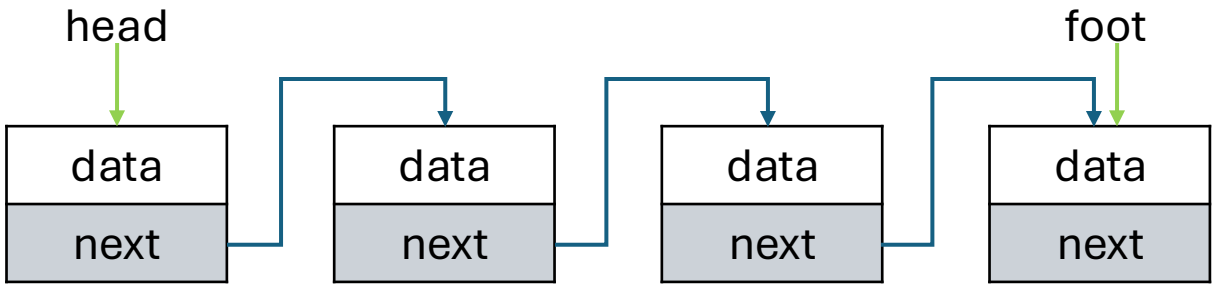
- A linked list is a one-dimensional data structure where each item is connected to the next one using a pointer in each node.

```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}
```



Linked List

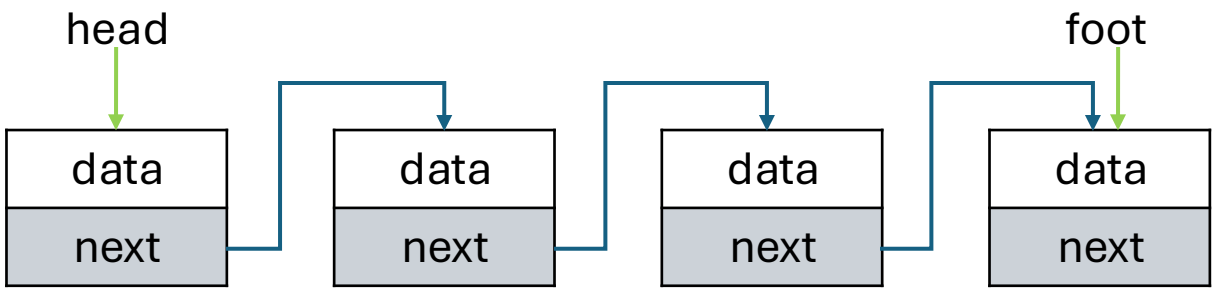
- A linked list is a one-dimensional data structure where each item is connected to the next one using a pointer in each node.




```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}

typedef struct {
        node_t *head;
        node_t *foot;
} list_t;
```

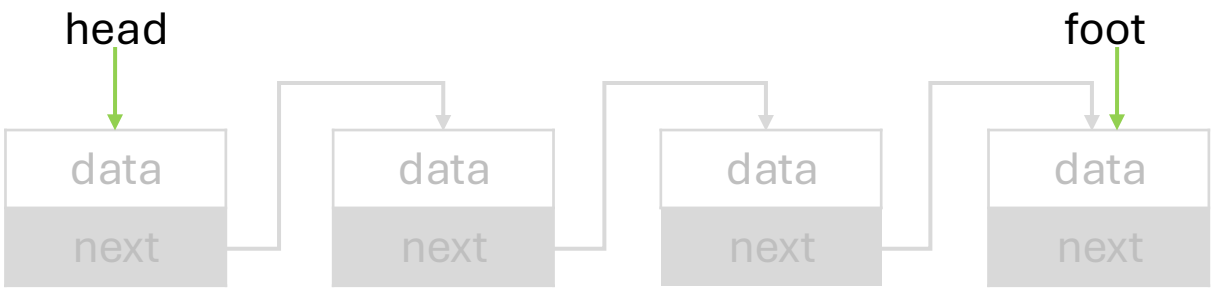
Linked List

- Creating an empty list



Linked List

- Creating an empty list

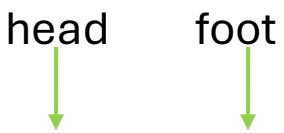


Linked List

- Creating an empty list
  (just 2 pointers – head and foot)

Linked List

- Creating an empty list
  (just 2 pointers – head and foot)




```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}

typedef struct {
        node_t *head;
        node_t *foot;
} list_t;
```

```
list_t
*make_empty_list(void) {
    list_t *list;
    list = (list_t*)malloc(sizeof(*list));
    assert(list!=NULL);
    list->head = list->foot = NULL;
    return list;
}
```

Linked List

- Creating an empty list
  (just 2 pointers – head and foot)

head foot

```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}

typedef struct {
        node_t *head;
        node_t *foot;
} list_t;
```

```
list_t
*make_empty_list(void) {
    list_t *list;
    list = (list_t*)malloc(sizeof(*list));
    assert(list!=NULL);
    list->head = list->foot = NULL;
    return list;
}
```

1. create an (empty) pointer to an object of the type list_t

Linked List

- Creating an empty list
  (just 2 pointers – head and foot)

head foot

```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}

typedef struct {
        node_t *head;
        node_t *foot;
} list_t;
```

```
list_t
*make_empty_list(void) {
    list_t *list;
    list = (list_t*)malloc(sizeof(*list));
    assert(list!=NULL);
    list->head = list->foot = NULL;
    return list;
}
```

1. create an (empty) pointer to an object of the type list_t
2. request enough memory to hold a list structure

Linked List

- Creating an empty list
  (just 2 pointers – head and foot)

head foot

```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}

typedef struct {
        node_t *head;
        node_t *foot;
} list_t;
```

```
list_t
*make_empty_list(void) {
    list_t *list;
    list = (list_t*)malloc(sizeof(*list));
    assert(list!=NULL);
    list->head = list->foot = NULL;
    return list;
}
```

1. create an (empty) pointer to an object of the type list_t
2. request enough memory to hold a list structure (this returns a pointer to the address of the allocated memory of type void)

Linked List

- Creating an empty list
  (just 2 pointers – head and foot)

head foot

```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}

typedef struct {
        node_t *head;
        node_t *foot;
} list_t;
```

```
list_t
*make_empty_list(void) {
    list_t *list;
    list = (list_t*)malloc(sizeof(*list));
    assert(list!=NULL);
    list->head = list->foot = NULL;
    return list;
}
```

1. create an (empty) pointer to an object of the type list_t
2. request enough memory to hold a list structure (this returns a pointer to the address of the allocated memory of type void)
3. Convert this pointer into list type

Linked List

- Creating an empty list
  (just 2 pointers – head and foot)

head foot

```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}

typedef struct {
        node_t *head;
        node_t *foot;
} list_t;
```

```
list_t
*make_empty_list(void) {
    list_t *list;
    list = (list_t*)malloc(sizeof(*list));
    assert(list!=NULL);
    list->head = list->foot = NULL;
    return list;
}
```

1. create an (empty) pointer to an object of the type list_t
2. request enough memory to hold a list structure (this returns a pointer to the address of the allocated memory of type void)
3. Convert this pointer into list type
4. Assign the pointer list to the address of the newly allocated memory

Linked List

- Creating an empty list
  (just 2 pointers – head and foot)

head foot

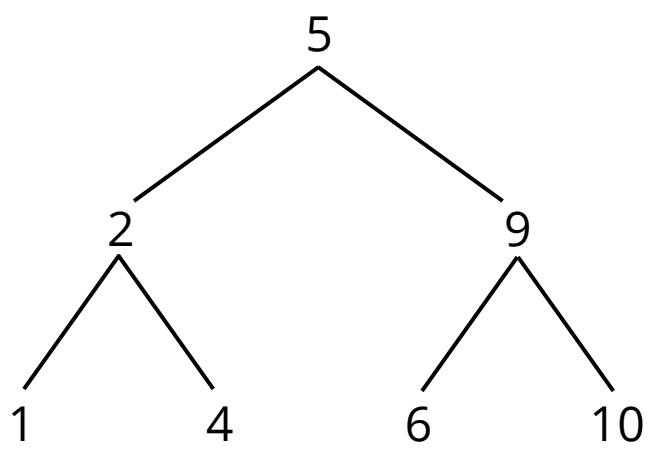
```
typedef struct node node_t
struct node{
    data_t data;
    node_t *next;
}

typedef struct {
        node_t *head;
        node_t *foot;
} list_t;
```

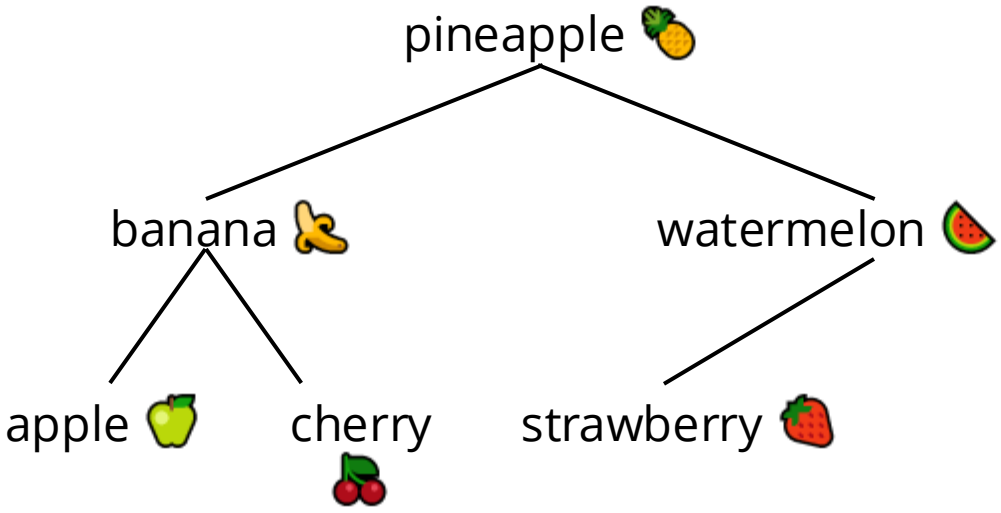
```
list_t
*make_empty_list(void) {
    list_t *list;
    list = (list_t*)malloc(sizeof(*list));
    assert(list!=NULL);
    list->head = list->foot = NULL;
    return list;
}
```

1. create an (empty) pointer to an object of the type list_t
2. request enough memory to hold a list structure (this returns a pointer to the address of the allocated memory of type void)
3. Convert this pointer into list type
4. Assign the pointer list to the address of the newly allocated memory
5. Double-check whether the memory allocation worked correctly

Binary Search Tree

left value < root < right value





Ex10.08: Draw a BST

Ex10.08: Draw a BST

Construct (draw) a binary search tree on strings by inserting the signs of the Zodiac in their sequence order into a tree that is initially empty: Aquarius, Pisces, Aries, Taurus, Gemini, Cancer, Leo, Virgo, Libra, Scorpio, Sagittarius, Capricorn.

Then add in the twelve animals of the Chinese zodiac: Rat, Ox, Tiger, Rabbit, Dragon, Snake, Horse, Goat, Monkey, Rooster, Dog, Boar.

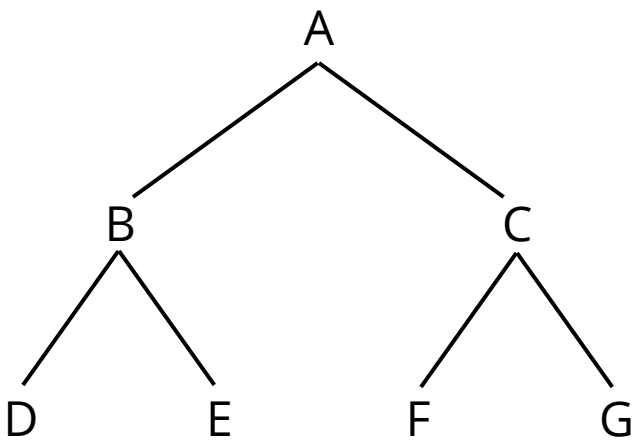
Then add in your name and the name of some friends.

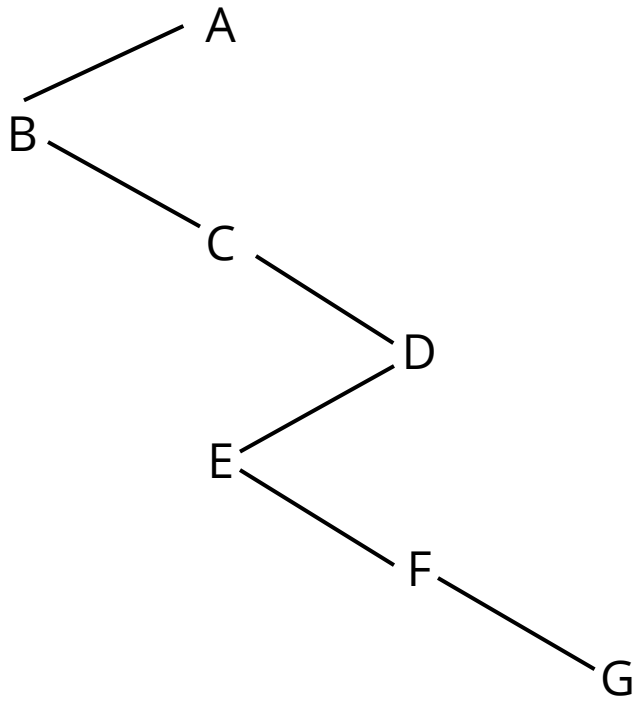
Finally, calculate the average depth of the items in the tree, with the root node of the tree at depth 1, its children at depth 2, and so on.

Verify that for "typical" input sequences like this one, the average depth of the items in the tree is much smaller than the number of objects in the tree.

Binary Search Tree

- Time complexities of ordered **insert, delete** and **search** for a binary search tree:
  - Best case/Average case (balanced tree): $O(\log n)$
  - Worst case (stick): $O(n)$



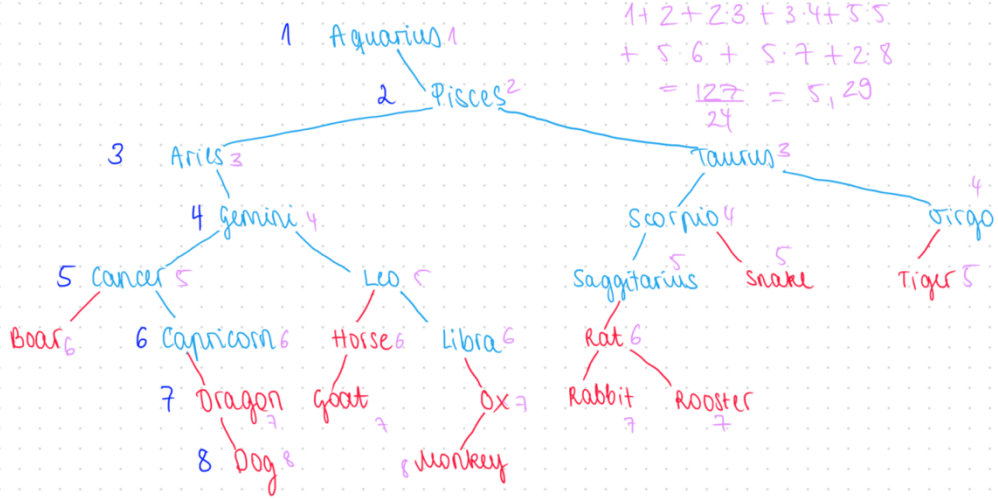


indiv. depths

$1 + 2 + 2 \cdot 3 + 3 \cdot 4 + 5 \cdot 5 + 5 \cdot 6 + 5 \cdot 7 + 2 \cdot 8 = \frac{127}{24} = 5{,}29$

24 entries

but only depth of 8



Time complexities of insert, delete, search for BST?

all: $O(\log(n))$ average / best case (balanced tree)

$O(n)$ worst case (stick)

56%

merge sort